
用户信息管理平台

安全漏洞挖掘与修复报告

http://192.168.43.129:5000

项目版本： V3.0 — 安全加固版（密码安全专题）

报告日期： 2026年7月7日

今日课程： 密码安全 — 硬编码密码 / 暴力破解 / 明文存储

技术栈： Python Flask / SQLite / bcrypt

安全评级： *****

一、漏洞分析（密码安全专题）

根据今日课堂密码安全专题内容，对系统进行白盒审计，共发现 7 项与密码直接相关的安全漏洞（含 2 项极高危、3 项高危、2 项中危）。以下逐条分析：

[高危] 极高危 V-01 硬编码默认密码

app.py 中直接将管理员密码 "admin123" 和普通用户密码 "alice2025" 以明文形式硬编码在 USERS 字典中。同时 login.html 第 1 行的 HTML 注释也写明了管理员账号密码。任何人查看源码即可获得登录凭据。

-> 攻击后果: 攻击者直接获得管理员权限，完全控制系统

```
# [x] 修复前：硬编码明文密码
USERS = {
    "admin": {"password": "admin123"}, # 明文硬编码!
    "alice": {"password": "alice2025"} # 明文硬编码!
}

# 同时 login.html 注释泄露:
<!-- 调试信息 - 默认管理员账号 用户名: admin 密码: admin123 -->
```

[高危] 极高危 V-02 密码强度不足 & 无爆破防护

密码 "admin123" 和 "alice2025" 均为弱密码，仅包含小写字母和数字，长度不足 10 位，且无任何特殊字符。同时登录接口无速率限制，攻击者可用 Burp Suite 等工具进行字典爆破。

-> 攻击后果: 弱密码 + 无防护 = 短时间内即可被暴力破解

```
# [x] 修复前：无速率限制
if username in USERS and USERS[username]["password"] == password:
    session["username"] = username # 直接登录，无任何限制

# Burp Suite Intruder 可以每秒数百次尝试密码
```

[高危] 高危 V-03 密码明文存储与比对

密码以明文形式存储在字典中，登录验证使用 == 直接字符串比对。未使用任何哈希算法，数据库泄露即意味着所有密码完全暴露。

-> 攻击后果: 所有用户密码凭据完全泄露，影响所有账户

[高危] 高危 V-04 密码在前端页面明文展示

index.html 中使用 {{ user.password }} 直接在页面上渲染密码字段，登录后的用户信息页面完整显示密码明文。

-> 攻击后果: 登录后密码持续暴露在浏览器端

```
# [x] 修复前：密码显示在前端
<p><strong>密码: </strong>{{ user.password }}</p> # 密码直接输出!

# [OK] 修复后：过滤密码字段
user_info = {k:v for k,v in raw.items() if k != "password"}
```

[高危] 高危 V-05 弱 Secret Key 导致 Session 可伪造

app.secret_key = "dev-key-2025" 为简单字符串，攻击者可利用此密钥伪造 Session Cookie，无需密码即可冒充任意已登录用户。

-> 攻击后果: 无需密码即可实现任意账户身份冒充

[中危] 中危 V-06 无 CSRF 防护可被利用修改密码

登录表单无

CSRF

Token，攻击者可伪造请求诱导用户修改密码。（虽然当前系统无修改密码接口，但存在潜在风险）

-> 攻击后果: 跨站请求伪造，可被用于密码重置攻击

[中危] 中危 V-07 缺乏登录失败审计日志

系统未记录任何登录成功/失败事件。发生密码爆破时，无法确定攻击来源、时间和攻击范围。

-> 攻击后果: 安全事件无法溯源分析

二、漏洞修复（密码安全加固）

针对上述密码相关的 7 项安全漏洞，实施以下 15 项专项加固措施。

【核心密码安全修复】

修复 1 bcrypt 密码哈希存储（V-01, V-03）

使用 `werkzeug.security.generate_password_hash()` 对密码进行 `bcrypt` 加盐哈希。登录验证使用 `check_password_hash()` 常量时间比对，防止时序攻击。密码无论存储在字典还是数据库中，均为哈希值而非明文。

```
# [OK] 修复后：bcrypt 哈希
from werkzeug.security import generate_password_hash, check_password_hash

# 存储时哈希:
_entry["password"] = generate_password_hash("admin123")

# 验证时用常量时间比对:
if check_password_hash(stored_hash, input_password):
    # 密码正确
```

修复 2 删除硬编码密码 + 清理注释（V-01）

从代码中移除所有明文密码字面量，改用哈希值存储。删除 `login.html` 中泄露凭据的 HTML 注释。用户数据通过 SQLite 数据库持久化存储，不再出现在源代码中。

修复 3 双重速率限制防爆破（V-02）

IP 级别限制：同一 IP 15 分钟内超过 5 次登录失败 → 返回 HTTP 429。用户级别限制：同一账户 15 分钟内超过 5 次失败 → 触发账号锁定。

```
# [OK] 双重速率限制
RATE_LIMIT_MAX = 5      # 最大尝试次数
RATE_LIMIT_WINDOW = 900 # 时间窗口 15 分钟

# IP 级限速
if ip_record["count"] >= RATE_LIMIT_MAX:
    return "登录过于频繁", 429

# 用户级锁定
if user_record["count"] >= RATE_LIMIT_MAX:
    user_record["locked_until"] = now + 900 # 锁定 15 分钟
```

修复 4 渐进式账号锁定（V-02）

第 1 次锁定 15 分钟 → 第 2 次 1 小时 → 第 3 次起 24 小时。登录成功后自动重置锁定计数器。每次锁定在审计日志中记录。

```
# [OK] 渐进锁定
_LOCKOUT_DURATIONS = [15, 60, 1440] # 分钟
idx = min(lockout_count - 1, 2)
duration = _LOCKOUT_DURATIONS[idx] * 60 # 转秒
```

修复 5 前端移除密码展示（V-04）

用户信息字典在传递给模板时过滤掉

`password`

字段。`index.html`

中删除密码渲染代码，余额、手机号等信息正常展示。

修复 6 随机 Secret Key 防 Session 伪造 (V-05)

使用 secrets.token_hex(32) 生成 256 位随机密钥，支持通过环境变量 SECRET_KEY 注入，不硬编码在代码中。

修复 7 CSRF 令牌防密码篡改 (V-06)

登录表单添加 `_csrf_token` 隐藏字段，使用 `hmac.compare_digest()` 进行常量时间比较。令牌一次性失效，防止重放攻击。

```
# [OK] CSRF 防护
<input type="hidden" name="_csrf_token" value="{{ csrf_token }}">

def _validate_csrf_token():
    return hmac.compare_digest(token, stored)
```

修复 8 审计日志记录密码事件 (V-07)

使用 `RotatingFileHandler` 轮转日志 (5MB)，记录所有与密码相关的安全事件：登录成功/失败、账号锁定/解锁。每条日志包含时间戳、IP 地址、用户名和 User-Agent。

```
# [OK] 审计日志样例
[2026-07-07 04:33:16] IP=127.0.0.1 USER=admin ACTION=LOGIN_SUCCESS
[2026-07-07 04:34:41] IP=127.0.0.1 USER=admin ACTION=ACCOUNT_LOCKED
RESULT=LOCKED duration=15min count=1
```

【增强型安全措施】

修复 9 Session 指纹绑定

将登录时的 IP + User-Agent 通过 HMAC-SHA256 生成指纹存入 Session。每次请求校验指纹，防止 Session 劫持后密码被绕过。

修复 10 Session 双过期机制

30 分钟无操作自动过期 (滑动过期) + 24 小时强制重新登录 (绝对过期)。确保长期未操作的会话不会泄露密码信息。

修复 11 Cookie 安全属性

`SESSION_COOKIE_HTTPONLY=True` (禁止 JS 读取 Cookie)、`SESSION_COOKIE_SAMESITE="Strict"` (严格同站策略)。

修复 12 安全响应头

配置 CSP、X-Frame-Options: DENY、X-Content-Type-Options 等 8 个安全头。

修复 13 SQLite 数据库持久化

将用户数据从内存字典迁移至 SQLite 数据库。密码字段存储 bcrypt 哈希值，数据库文件通过文件权限保护。

```
# [OK] SQLite 用户表
CREATE TABLE users (
  username TEXT PRIMARY KEY,
  password TEXT NOT NULL, -- bcrypt 哈希
  role TEXT, email TEXT,
  phone TEXT, balance INTEGER,
  lockout_count INTEGER, -- 锁定计数
  last_login TEXT      -- 上次登录
);
```

修复 14 蜜罐字段防自动化

登录表单添加 `CSS` 隐藏的 `_gotcha` 字段，自动化脚本会自动填写，一旦有值即拒绝请求，防止自动化密码爆破工具。

修复 15 统一错误提示防枚举

无论用户名不存在还是密码错误，均返回「用户名或密码错误」。防止攻击者通过错误信息差异枚举有效用户名。

三、修复结果检测

修复完成后进行安全测试验证，以下是针对密码安全维度的检测结果：

- 1. 用原来的字典，已经爆破不到密码了
→ bcrypt 哈希有效抵御字典攻击和彩虹表攻击
- 2. 源码中账密不可见
→ 硬编码凭据已删除，密码以哈希值存储在 SQLite 数据库中
- 3. Debug 模式仅当 FLASK_DEBUG=1 时开启
→ 默认关闭调试器，防止密码信息通过调试页面泄露
- 4. 暴力破解已被有效阻止
→ 第 6 次错误登录即返回 HTTP 429，账号锁定 15 分钟
- 5. 前端不再展示密码
→ 登录成功后页面无 password 字段渲染
- 6. CSRF Token 有效防护
→ 无 Token 的 POST 请求被 HTTP 400 拒绝
- 7. 审计日志完整记录
→ 每次登录失败/账号锁定均在 logs/audit.log 中有记录

密码安全专项测试（7 项全部通过）

序号	测试项	测试方法	结果
1	硬编码密码检测	搜索源码中的 password 字段	[OK] 已全部移除
2	密码哈希验证	查看数据库存储内容	[OK] bcrypt 哈希值
3	暴力破解防御	连续 6 次错误登录	[OK] HTTP 429 限速
4	账号锁定	5 次错误后尝试登录	[OK] HTTP 423 锁定
5	前端密码泄露	登录后查看页面元素	[OK] 无 password 字段
6	CSRF 防护	无 Token 的 POST 请求	[OK] HTTP 400 拒绝
7	审计日志	检查 audit.log 文件	[OK] 完整记录

四、总结

本次安全加固聚焦密码安全专题，针对课堂所学的 3 大密码风险进行了全面修复：

课堂知识点	对应漏洞	修复措施
1. 硬编码密码	V-01 硬编码默认密码	删除明文硬编码 + bcrypt 哈希存储 + SQLite
2. 密码爆破	V-02 密码弱 + 无限速	双重速率限制 + 渐进式账号锁定
3. 密码泄露	V-03/V-04 明文存储/展示	bcrypt 哈希 + 前端过滤 + 安全响应头

额外加固措施：Session 指纹绑定、CSRF 防护、审计日志、蜜罐字段、统一错误提示等项增强防护，构建了密码安全的纵深防御体系。

— 报告完 —